

06. AI 学习之路 · 实战篇之「为什么要自己搭一个 AI 知识库」

来自：帅张和他的朋友们

 stormzhang

2026年06月02日 15:11

扫码
查看更

兄弟们，从今天起，我们正式开始更新「AI 学习之路」系列的实战篇——知识库。如果你还没什么 AI 基础，建议先抽时间，把之前更新的 AI 基础系列看完。

实战篇会带你走一整套产品开发流程——产品思维、工程架构思维、技术思维，再到 Vibe Coding 阶段。中间的实操环节，我觉得有必要的话，会面向球友做单独的私密直播答疑。跟着我的脚步走完这套系列，每个人都能借助 AI，做出一款属于自己的最小可用产品。

那么废话不多说，第一篇正式开始。

我们都干过同一件事。

看到一篇好文章，点收藏；刷到一个好观点，截图；遇到一个好工具，顺手存进 Notion 之类的笔记 App。手指一按，心里就踏实了——感觉这点知识，已经归你了。

然后呢？然后就没有然后了。收藏夹越堆越满，Notion 里建了十几个数据库，可真要找东西，你还是在微信搜索框里大海捞针。更扎心的是：你对任何一个主题的理解，并没有因为「收藏了 50 篇」就变深一点。要用的时候，还得从头翻、从头读、从头想。说白了，你存下来的是信息，不是知识。

这个毛病，2026 年 4 月被一个 AI 大神——卡帕西——用一种很反直觉的方式治了。我照着他的思路搭了一套自己的知识库，用了一段时间，又在他的设计上动了刀。是的，你没看错：我这套知识库，比卡帕西的多了几个他没有的功能，后面会详细讲。

这篇是「AI 学习之路 · 实战篇」的开篇，咱们先不碰任何代码，先把一件最基础的事聊明白：到底什么才配叫「知识库」，以及为什么你该自己搭一个。

01 收藏夹、Notion、Obsidian，为什么都不够用

先问一个问题：你现在的「知识」都存在哪？

我猜一下，大概散落在七八个地方：微信收藏、浏览器书签、备忘录、Notion、Obsidian、各种聊天记录，还有一大部分在你自己脑子里——而脑子这个存储，你我都清楚，不太靠谱。

这些工具，存东西都没问题。问题是它们都卡在同一个天花板上：存得下，但不「懂」你存的东西。

拿大家最常用的几个说：

- **微信收藏 / 浏览器书签**：纯粹的「往后堆」。收藏了就收藏了，条目和条目之间没有任何关系，时间一长就是个黑洞。
- **Notion**：数据库做得再花哨，本质还是你自己整理、自己填表、自己分类。它是个特别精致的仓库，但搬货的还是你。
- **Obsidian**：双向链接这个概念很酷，但链接得你手动一条条建。说句实话，这玩意儿能坚持手动连链接超过两周的人不到一成，我自己就是那九成里的。

更要命的是，它们的搜索几乎都是**关键词匹配**：你得记得当初存的时候用了什么词。记不住，就等于没存。你搜「提效」，结果当时记的是「效率」，这条就再也找不回来了。

类比：这些工具是仓库，不是助理。 仓库帮你把东西堆着，但不会帮你用；真正值钱的，是那个读过你所有资料、随时能帮你调出来、还能帮你一起想的助理。我们缺的从来不是仓库，是助理。

而所有这些工具的共同病根是同一个：它们永远停在「原始信息」这一层，从不帮你往上加工一步。

02 卡帕西给的答案：把知识「编译」一次，永久复用

那个治这毛病的人，叫 Andrej Karpathy（卡帕西）。

相信关注我公号的文章，对卡帕西大神不陌生。简单介绍下，这人是前特斯拉 AI 总监、OpenAI 的创始成员，深度学习圈里教科书级别的人物——很多人入门 AI 看的就是他的课。2026 年 4 月，他发了一条帖子，提出一个叫「LLM 知识库」（也叫 LLM Wiki）的玩法，第二天还甩出一份能直接喂给 Claude Code、Codex 的「配方文档」。

在那之后，市面上但凡提到知识库的设计或者 Skill，几乎都是照着卡帕西这套理念来的。

他的出发点就很有意思。这两年大家用大模型，主要是让它写代码；他想换个花样，让大模型来帮你攒知识。

一句话概括他的设计：

把你关于某个领域的所有原始资料，全丢进一个文件夹，让大模型把它们「编译」成一个互相链接的笔记库。从此这个笔记库就是你的「第二大脑」——能回答复杂问题，还会随你每次喂新资料，自己长大。

听着好像也就那样？关键全藏在「怎么组织」和「怎么运转」这两件事里。

先看组织。整套系统其实就三层：

- **raw（原料层）**：论文、文章、网页、PDF、截图，原始资料全丢这儿。大模型只读不改，这是唯一的「事实来源」。
- **wiki（成品层）**：大模型读完原料，在这儿写出整理好的、结构化的条目。这是整个系统的成品区。
- **schema（规矩层）**：一份配置文档，告诉大模型这个库该怎么组织、遵守什么规矩、走什么流程。

再加一个索引文件（index）：全库的目录，每个页面一行——一个链接加一句话摘要。你提问的时候，大模型先读这个索引，定位到相关页面，再钻进去看细节。

类比：这就像一本书的目录加正文。原料是你买来的一摞参考书，wiki 是你自己消化后写的读书笔记，索引是这本笔记的目录。要查东西，先翻目录，再翻到那一页——而不是每次都把那一摞参考书从头读一遍。

记住「编译」这个词，它是卡帕西整套理念的灵魂。下面这两点，才是这套设计真正妙的地方。

03 最妙的两点：它会「往回织」，还不用 RAG

第一点：加一条新资料，它会「往回织」，而不是「往后堆」。

这是它和收藏夹的本质区别。

假设你丢进一篇关于「Claude Code」的新文章。收藏夹的做法是：存一下，完事，新增一条，跟旧的东西毫无关系。

卡帕西这套不一样，它会走一套流程：读完这篇，先跟你聊聊关键要点；在 wiki 里写一个这篇的摘要；更新索引；然后——回头把所有相关的旧条目都更新一遍：「Claude Code」这一页补上新信息，「AI 编程工具」这个主题页加进新的对比，甚至发现这篇和你之前存的资料有矛盾，还会专门标出来。

卡帕西说，加一篇资料，可能会牵动 10 到 15 个已有页面。

你品品这个差别：收藏夹是「往后堆」，堆成一座越来越高的孤岛；这套系统是「往回织」——每一条新信息，都被编织进你已有的知识网里。这才是「知识在沉淀」该有的样子。

第二点，也是最反直觉的：它不用 RAG。

这点得稍微解释下。这两年但凡聊「AI + 知识库」，几乎所有方案的标准答案都是 RAG：把你的文档切成小块、转成向量、存进一个专门的数据库，你提问的时候，它去临时捞出最相关的几块喂给 AI。

类比：RAG 像一个每次现去仓库翻原料的厨子。你点一道菜，他才跑进仓库，翻出相关食材，现切现炒。每次都从原料开始，重复劳动，还经常翻错、翻漏。

卡帕西说：不需要。他的理由是四个字——编译 > 检索。

与其每次提问都让 AI 重读一遍原始文档，不如一次性把它们编译成结构化的笔记，之后永久复用。

换句话说，他不是每次现翻原料，而是提前把原料炼成了半成品：你问问题时，直接读炼好的成品加目录导航，又快又准。他自己的库攒到差不多 100 篇、40 万字，光靠一个索引文件就能导航，他说仍然比 RAG 更快、更准。



为什么这个思路是对的？因为它戳中了知识的本质：

知识，是被反复加工、彼此连接之后的产物，不是原始信息的堆叠。

RAG 把「理解」这件事，推迟到了你每次提问的那一刻现做，所以它永远在重复劳动。卡帕西反过来，把「理解」前置成一次性的「编译」，沉淀下来，后面一直坐享其成。而收藏夹的毛病也正在这：它永远停在原始信息那一层，从不向上编译。

最爽的是，这套东西会**复利**：你越用，它越聪明，而不是越用越乱。

04 但你不能直接抄：为什么要「专属自己」

讲到这儿，你可能想：那好啊，我把卡帕西那份配方拿来，照着搭一个不就行了？

我一开始也这么想。照着搭完，用了一段时间，我发现一个问题：**卡帕西给的是一副通用骨架，但每个人要的知识库，根本不一样。**

差别在哪？在「输入」、「发散」和「终点」这三个地方。

先说「输入」。卡帕西的理念是，输入主要依靠源文件——你看过的 PDF 文档、论文、网页，扔给它，它去分析、存储。

但我在实际工作里，输入来源更多是外部链接：一个 GitHub 开源项目、一条 X 推文、一个 YouTube 视频。我把链接扔给 AI，它就该自动抓取、总结里面的内容，整理入库。

再说「发散」。我随时随地会蹦出一些脑洞和产品想法，为此我给知识库对接了微信——想法一来，就借助 AI 一步步追问、发散，再把整个思考过程存进库里。试想一下，过了一个月，我早忘了当时纠结过什么，但这套记录把当时的想法、发散和疑问全留了下来。某种意义上，这才真正算得上我的「第二大脑」。

最后是「终点」。卡帕西这套的终点是「**答得出**」——你问它任何问题，它能基于全库给你准确回答，这对研究者、工程师来说已经完美。但我不一样：除了是技术 & 产品人，我还有一部分工作是自媒体写作。我要的根本不是答案，是**选题和文章**，我的终点是「**写得出**」。一个只会回答问题的知识库，对我来说还差最后一公里——它知道得很多，却不会主动告诉我「你今天能写哪篇」。

你看，同样一个「知识库」，研究者要和写作者要的，完全是两个东西。

这就引出了这个系列最想教你的第一个思维——**产品思维**：

做一个产品，不是从「别人怎么做」开始，而是从「我到底要解决什么问题」开始。

卡帕西的配方再好，它解决的是卡帕西的问题。你要搭的那一套，得解决**你的**问题。你是做销售的，可能要的是随时调出客户资料和话术；你是做研究的，要的是文献之间的脉络；你是做自媒体的，像我，要的是从素材里直接长出选题。

照搬永远搭不出最趁手的工具。 所以这个实战篇从头到尾就围绕一件事：带你搭一个**属于你自己的**知识库，而不是复刻谁的。

05 我们的版本：在卡帕西之上，多想了几层

那我自己那套（我给它起了个项目代号叫 fuxi），到底在卡帕西的基础上动了什么刀？这里先给你看个大概，后面每一篇会拆开细讲。

地基是完全一样的：本地、纯 Markdown 文件、不依赖任何云服务、Claude 负责抓取和提炼、脚本负责索引和检索、坚决不用 RAG。

不一样的，是我在他「答得出」的终点之上，往「用得出」又推了几层。挑几个最关键的说：

一、输入层支持各种渠道源

这听起来简单，真正实现起来却有门槛：单说按链接抓取，微信公众号、GitHub、X、YouTube，每个渠道的抓取方式和技术难度都不一样。

二、摘要是有立场的，不做中性复述。 一般知识库的摘要，追求「客观概括」。我的不是。我的每一条摘要都得回答一个问题：「**这对我有什么用？**」 它会直接带判断——这个工具值不值得用、这个观点有没有新东西、甚至直接写「不值得

收」。因为知识库是我的「第二大脑」，第二大脑就该带着我的立场，不是一台复读机。

三、知识能直接「产出」，不止于回答。 这是我加的最关键的一层：库不光能回答问题，还能主动吐出「可以写的选题」。这一步，把「我知道很多」直接变成了「我今天能写哪篇」。

四、定期知识库周报 我们不能只是一味的输入，我觉得每周都要定期回顾我们的知识库增加了什么内容，方便我们复盘加深印象。

把这几点和卡帕西的版本摆一起对比，差别就很清楚了：

	卡帕西的版本	我们要做的版本
输入	依赖源文件	多渠道（公众号 / GitHub / X / YouTube）
终点	答得出（问答）	用得出（产出选题）
摘要	客观概括	带个人立场和判断
回顾	无	每周自动周报，定期复盘
服务对象	通用	为你的具体场景定制
语言场景	英文为主	中文、贴中文使用习惯

⚠️ 提前说清楚：**不是要你一上来就做出我这套**。fuxi 是我跑了挺久、迭代了很多轮的结果。这个系列会带你从一个最小、能跑起来的版本起步，理念上吸收卡帕西和我的这些经验，然后一篇篇往上加。先能用，再好用。

看到这里，如果你都理解了，应该就认同我开头说的那句——我这套知识库，确实在卡帕西的基础上多做了一些东西。

06 我们最终，要做出个什么东西

把话收回来。这个系列从头到尾，会带你亲手做出一个**属于你自己的、纯本地的 AI 知识库**。

先不谈技术怎么实现（那是后面选型和实战篇的事），先说做出来之后，它用起来是什么体验：

- **能存**：你看到的好文章、好观点、随手的灵感，丢给它就行，它帮你整理、提炼、归档。
- **能问**：你用大白话问它「我之前存的关于 XX 的东西，有哪些值得再看的」，它能理解你的意思，跨越几十条资料给你一个综合回答，而不是甩给你一堆关键词搜索结果。
- **是你的**：所有数据都在你自己电脑上，不上传任何云，不需要订阅，不被任何 App 绑架。它会按你的习惯，越长越懂你。

而做出它的路，我把它拆成了清清楚楚的四步，也就是这个系列的主线：

- 1 **产品思维**：先想清楚你的知识库要解决什么问题，砍出一个最小可用版本。
- 2 **架构思维**：把这个想法，拆成一张「东西从哪进、存哪、怎么取」的系统图。
- 3 **技术思维**：让 AI 帮你调研、选出最适合你的技术，这一步才真正定下用什么搭。
- 4 **Vibe Coding 实战**：用 Claude Code / Codex，把前面想好的东西，一步步变成能跑的真家伙。

注意一个反常识的安排：**前面三步，我们一行代码都不写**。

第四阶段 Vibe Coding 我们依然不写一行代码，但我们会通过自然语言指导 AI 帮我们写代码。

这也是我做了这么多东西最大的体会——普通人做产品最容易翻车的地方，从来不是不会写代码，而是还没想清楚要做什么，就急着开干。AI 时代，敲代码这件事基本被 AI 接管了，真正稀缺的，是「想清楚」的能力。

所以这个系列，本质上不只是教你搭一个知识库，是带你完整走一遍「用 AI 把一个想法变成产品」的全过程。知识库，只是我们练手的那个东西。

动手：先盘一盘，你的知识都散在哪

老规矩，每篇都有能上手的环节。这一篇不写代码，但有个更重要的功课——**搞清楚你自己到底要什么**。这直接决定了你下一篇会设计出一个什么样的知识库。

花十分钟，拿张纸或者开个空白文档，回答三个问题：

1. **我的知识，现在都散落在哪几个地方？** 微信收藏、书签、Notion、备忘录、相册截图、某个文件夹……一个个列出来。列完你大概会被自己吓到——原来摊了这么多摊子。
2. **我最常需要、却最难找回来的，是什么类型的东西？** 是看过的文章观点？是收集的工具？是聊天里闪过的灵感？还是工作上的某类资料？圈出最让你头疼的那一类。
3. **如果只能让这个知识库帮我解决一个问题，我最想要它帮我做什么？** 是「让我随时能找回存过的东西」？是「帮我把零散的想法串成体系」？还是像我一样「从素材里长出能用的产出」？

把这三个答案写下来，留好。下一篇讲「产品思维」，我们要做的第一件事，就是把你这三个答案，变成一份一页纸的产品定义。

小提示：这一步别偷懒，也别问 AI。这三个问题的答案必须是你自己的，因为这是整个知识库的地基。地基歪了，后面搭得再漂亮也是别人的房子。

小结

这一篇是开篇，主要搞定了最重要的认知地基：

- 你的收藏夹是「死」的——它停在「信息」这一层，从不帮你加工成「知识」。
- 卡帕西给了一个好答案：把知识编译一次、永久复用，每条新资料都「往回织」进已有的网里，而且不用 RAG，越用越聪明。
- 但你不能照抄他的配方，因为**每个人要的知识库不一样**——这正是「产品思维」的起点。
- 我们要做的，是一个属于你自己的、纯本地的版本，在卡帕西之上，让知识不只是「答得出」，还能「用得出」。
- 做它的路分四步：产品 → 架构 → 选型 → 实战，前三步不写一行代码，先把「想清楚」练扎实。

下一篇，我们正式动工第一步——**产品思维**。我会带你把今天写下的那三个答案，砍成一份真正能落地的「一页纸 MVP 定义」。你会发现，做产品最难的不是加功能，是狠下心做减法。

本系列持续更新，建议收藏。